

inc-mb-fidelity-inc___fidelity-binomial-f-k4-inc

An AgentMPI run analysis

Abstract

This is the depth-3 cell of the only configuration in the whole reduction-fidelity experiment that actually loses information: eight ranks, twelve high-entropy items each, a 450-token merge budget *enforced* by contract rather than suggested in a prompt. Retention is 0.385, and the aggregate is the least interesting thing about it. Ranks 0, 1 and 2 are retained completely, rank 3 keeps one item of twelve, and ranks 4 through 7 contribute nothing whatever. I recovered every intermediate from the fabric, and the mechanism is visible in a single blob digest: the root's final merge, which was handed the 37-item accumulator for ranks 0–3 and the 38-item accumulator for ranks 4–7, returned result `9f300f3a45bf` — byte-for-byte its own left input, 437 tokens, 37 identifiers. Half the population was discarded in one application that satisfied its contract and reported success. The reason is arithmetic rather than algorithmic: an incompressible item costs 11.81 tokens in situ, so a 450-token ceiling admits 37 or 38 of them and no more, and that ceiling binds at whichever application is last regardless of how many preceded it. This cell's three fold levels and the chain's seven and the k -ary tree's two all land on the same 37 survivors, so the tree-shape hypothesis is not merely unsupported here, it is refuted by construction of the budget.

1 What this run is

property	value
run	inc-mb-fidelity-inc___fidelity-binomial-f-k4-inc
experiment	-
campaign label	-
declared world size	8
ranks appearing in the log	8
executors	broker $\times$ 8, worker $\times$ 18
events	87
wall time	80.27 s
job outcome	completed

2 Headline measurements

metric	value	meaning
achieved parallelism	1.90×	total busy time over wall time
parallel efficiency	23.8%	achieved parallelism per rank
idle fraction	76.2%	rank-seconds paid for and unused
peak ranks busy	4	of 8 in the world
serial fraction of active time	45.3%	time with exactly one rank busy
load imbalance	2.94×	slowest rank’s busy time over the mean
coordination share	28.9%	rank-seconds blocked in collectives, of those available
coordination in flight	100.0%	wall clock with any rank inside a collective
rank reattachments	18	fresh agent processes that took over a rank role
agent calls	7	invocations that reached a model
agent latency p50 / p95	22.01 / 30.86 s	per invocation
messages	7	7 eager, 0 rendezvous
tokens sent	1,806	0 deferred under rendezvous
tokens in / out	4,220 / 2,492	prompt and completion
cost	\$0.0500	0.0201 \$/kTok out

3 What the analysis notices

- **[critical]** At least one rank waited 5s for an agent to claim its task before the broker gave up. That is a pool-sizing failure outside the protocol, not a slow run.
- **[note]** Parallel efficiency is 23.8%: 76.2% of the rank-seconds paid for went unused.
- **[note]** Load imbalance is 2.94×: the slowest rank was busy far longer than the mean.
- **[ok]** 18 rank reattachment(s) occurred, up to incarnation 6: agent processes were replaced mid-run while the rank roles, their mailboxes, and their context accounts persisted.
- **[ok]** All 1 checkable collective(s) sent exactly the number of messages their closed-form cost expression predicts.
- **[ok]** All 1 checkable collective(s) also matched the predicted round count.

4 Per-rank detail

rank	busy (s)	occ. (%)	tasks	calls	sent	recv	tok. sent	ctx (%)	state
0	56.16	49.9	3	3	0	3	0	0.0	finalized
1	0.00	0.0	0	0	1	0	178	0.0	finalized
2	25.24	80.6	1	1	1	1	312	0.0	finalized
3	0.00	0.0	0	0	1	0	178	0.0	finalized
4	51.75	45.0	2	2	1	2	469	0.0	finalized
5	0.00	0.0	0	0	1	0	178	0.0	finalized
6	19.57	78.8	1	1	1	1	313	0.0	finalized
7	0.00	0.0	0	0	1	0	178	0.0	finalized

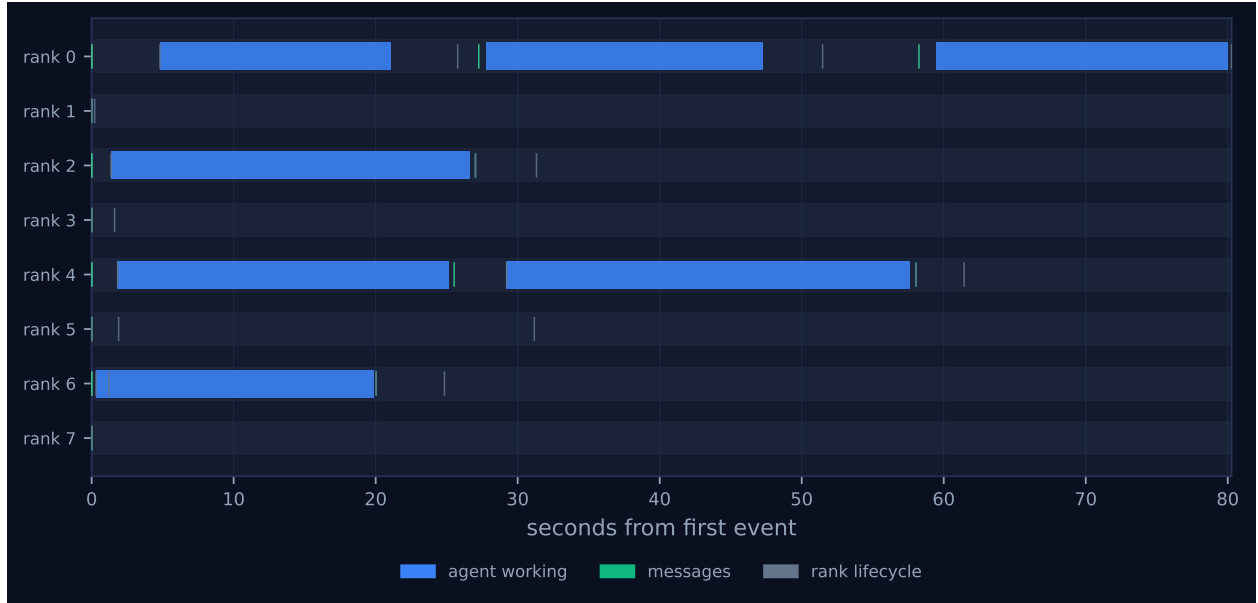


Figure 1: Execution timeline, one lane per rank. Bars are intervals when a rank held a task; ticks are messages; diamonds are window operations, lifecycle transitions, failures, and recovery actions. Drawn in the trace viewer's palette so the same shapes read the same way in both.

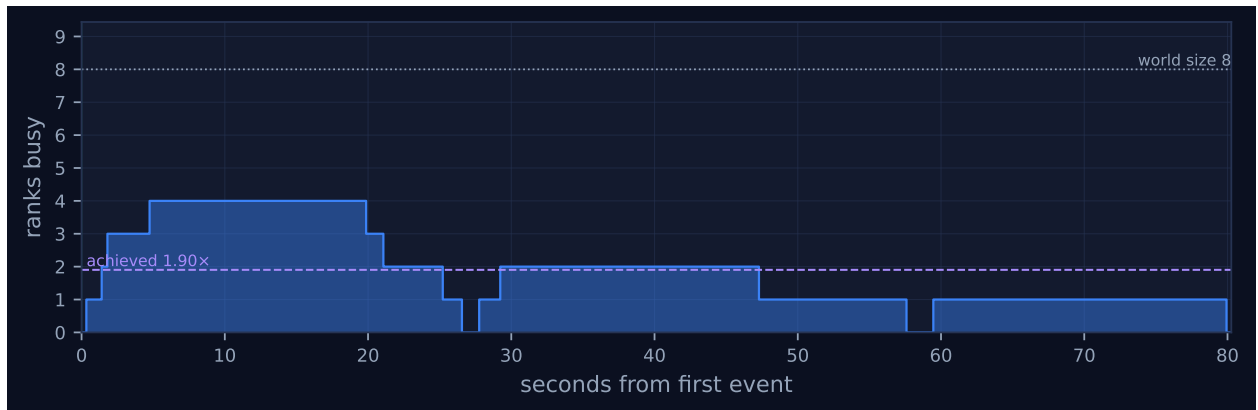


Figure 2: Ranks simultaneously busy over time. The dotted line is the declared world size and the dashed line the achieved parallelism; the gap between them is what the run paid for and did not use. Faint vertical lines mark collective invocations.

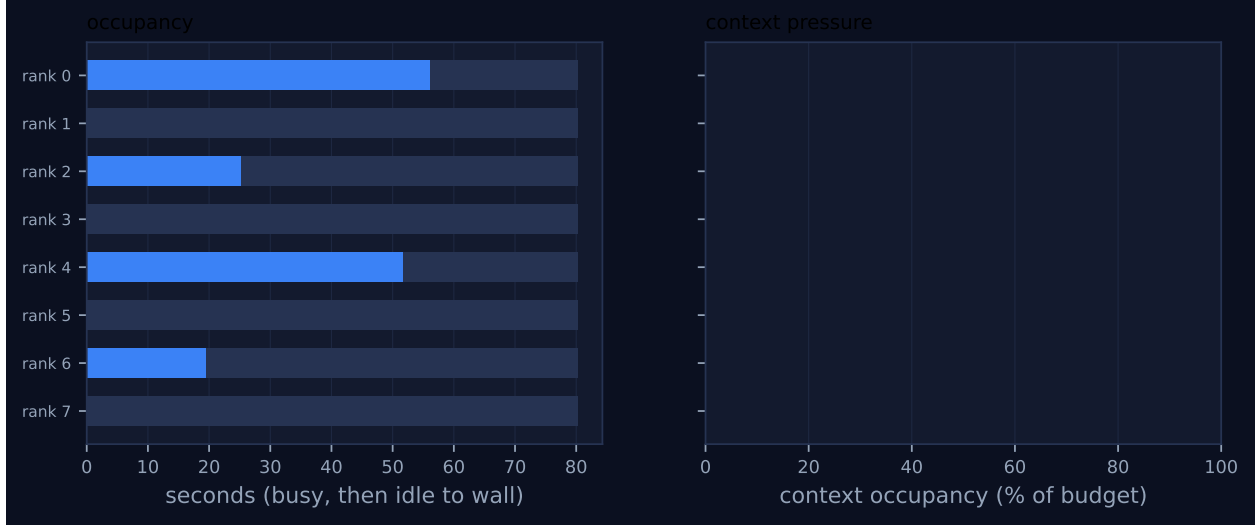


Figure 3: Per-rank occupancy and context pressure. Left: busy time, with the remainder to wall time in grey. Right: context occupancy against budget, amber above half and red above 80%, where admission pressure starts to reject artefacts.

Per-rank behaviour. Occupancy is busy time over the rank’s own lifetime, so a rank that joined late is not penalised for the time before it existed.

5 Collectives, against the cost model

op	algorithm	label	p	seen	rounds	pred.	msgs	pred.	tokens	wall (s)	skew (s)	model
reduce	binomial	–	8	8	3	3	7	7	1,668	80.26	80.26	✓

Messages are the count the fabric logged, attributed to each invocation through its internal tag, rather than the count the collective reported — the two are independently produced, and preferring the log is what makes this a check rather than a restatement. A dash in the model column means the comparison is not meaningful: either no closed form exists for that algorithm, or the invocation never completed.

6 Reading the timeline

Figure 1 is the textbook binomial tree drawn in wall clock, and it is the only cell in this campaign with real concurrency. Three bands of work: ranks 0, 2, 4 and 6 all busy together from about $t = 1$ s to $t = 26$ s, then ranks 0 and 4 together to about $t = 55$ s, then rank 0 alone to 80.27 s. Peak ranks busy is 4 of 8, achieved parallelism $1.90\times$, and the serial fraction of active time is 45.3% — the three-level structure means the last level is necessarily serial. Ranks 1, 3, 5 and 7 show one tick each within the first 20 ms and then nothing: they are the leaves of the tree, they send their twelve-item report to their parent and return, and their `coll.reduce` records `fold_depth: 0`. That is why load imbalance is $2.94\times$ and 76.2% of the rank-seconds paid for went unused; both are what a binomial tree at $p = 8$ must produce and neither is a defect.

The bars do not grow the way a naive reading would predict, and that is the run’s first substantive detail. The four depth-1 merges took 16.22, 19.57, 23.36 and 25.24 s on prompts of exactly 446

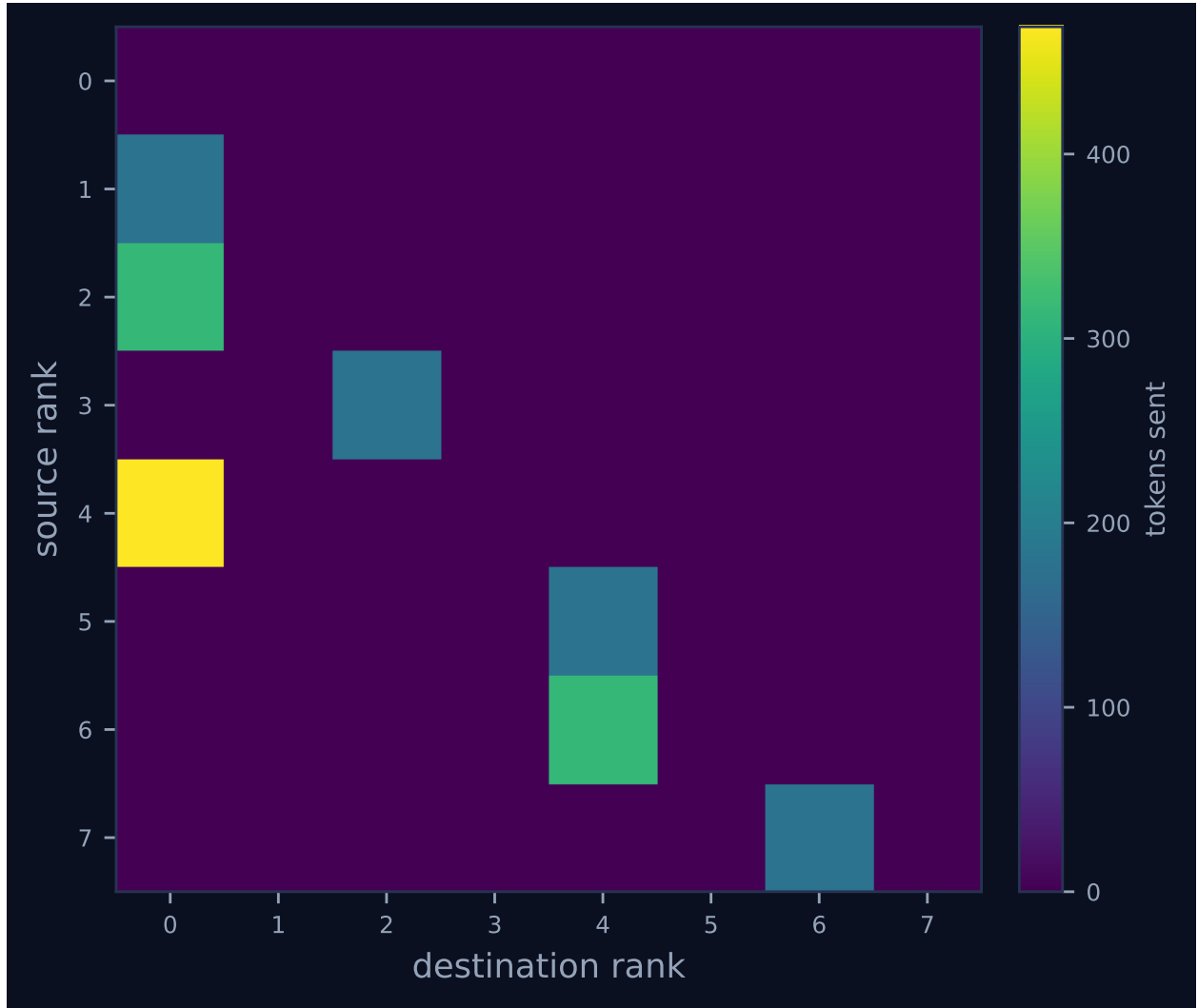


Figure 4: Communication matrix: tokens sent from each rank to each rank. The algorithm’s shape is visible here — a tree concentrates traffic on a root, a ring forms a diagonal band, and an unintended all-to-all fills the plane.

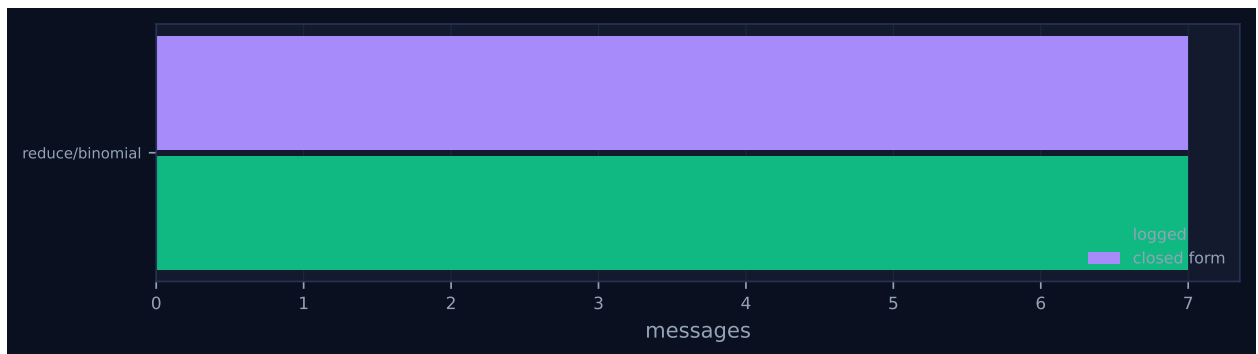


Figure 5: Logged message count against the closed-form prediction, per invocation. A red diamond marks a disagreement between the implementation and its own cost model.

tokens each; the two depth-2 merges took 19.45 and 28.39s on 708 and 714 tokens; the final depth-3 merge took 20.49s on 1,014 tokens. Latency is flat in prompt size across a $2.3\times$ range, because the output is pinned near the contract ceiling at every level — 288–294 tokens at depth 1, 437 and 449 at depths 2 and 3 — and generation, not prompt ingestion, dominates. The whole run cost 152.7s of generation and only 13.7s of waiting for a worker to claim a queued call (I decomposed each `broker.enqueue` $\rightarrow$ `broker.claim` $\rightarrow$ `broker.complete` triple from the log), so unlike its chain sibling this cell’s wall time is honest model time. The 18 reattachments and the log’s three events after `job.finish` at 80.27s are the worker pool churning after the job is over; the dashboard’s rank health table shows all eight ranks red and suspected `fail_stop` for that reason, while `metrics.json` records `ok: true` and no failures.

The communication matrix, Figure 4, is the tree’s adjacency: $1 \rightarrow 0$, $3 \rightarrow 2$, $5 \rightarrow 4$, $7 \rightarrow 6$ at 178 tokens each, then $2 \rightarrow 0$ at 312 and $6 \rightarrow 4$ at 313, then $4 \rightarrow 0$ at 469, for 1,806 tokens over seven messages. The edge weights are the story in miniature: a leaf carries 12 items in 178 tokens, a half-subtree carries 24 in 312, and the quarter-population edge $4 \rightarrow 0$ carries 38 in 469. Traffic grows sublinearly in the number of items represented because the accumulator is capped, and the last edge is already at the cap.

7 Interpretation

Everything structural here is by construction and both checkable predictions hold: seven messages against a closed form of $p - 1 = 7$, three rounds against $\lceil \log_2 8 \rceil = 3$, so `model_checks_agree` is 1 of 1 and Figure 5 carries no red diamond. What must be established before anything else is which enforcement regime this run is in, because the entire interpretation turns on it, and the fabric settles it. Every one of the seven `agent_calls` rows records `contract = {"name": "MergedReport", ..., "max_tokens": 450, ..., "semantics": "findings must contain complete, verbatim items..."}`. That is the enforced form: the runtime checks the returned token count, rejects an overrun, and re-issues the call with the diagnosis appended. Contrast the `sat-mb-fidelity` siblings, which ran the same $p = 8$, twelve-items, 450-token configuration with `max_tokens: null` and an empty semantics string, and scored 1.000 by overrunning the stated budget by up to 55%. The provenance block records commit `f121647`, in which `merge_contract` does not yet exist; the enforced contract was committed two minutes after this campaign wrote its results, as `561d8a17`, “A budget stated in a prompt is not a budget”. The commit field is HEAD at launch, not the working tree, and the fabric is the authority.

What emerged is that retention under an enforced budget is a division, not a property of the tree. An incompressible item is `[F-0-0] serial 597623 checksum LVPG`: nine tokens of content with no shorter equivalent, 11.81 tokens once quoted inside the JSON array. A 450-token ceiling therefore admits $450/11.81 = 38.1$ items, and every merge in this run lands exactly there — 437 tokens for 37 items and 449 for 38, at depths 2 and 3. The reduction never fails to fill its budget and never exceeds it; it simply stops. So the surviving count is fixed at 37 and the aggregate is $37/96 = 0.385$ before the algorithm is chosen. Reconstructing the fold from the store makes the consequence concrete. The two depth-1 merges under rank 0 return 24 identifiers each; their combination at depth 2 returns 37, so eleven of rank 3’s twelve items were already gone. The mirror subtree returns 38 for ranks 4–7. The final application is handed $37 + 38 = 75$ items, has room for 37, and returns its left input unchanged — same digest, same 1,444 bytes. Rank 3’s single survivor is `F-3-0`, the first of its twelve, so the result is a strict prefix of the rank-ordered item list truncated at the budget. This is why `rank_position_correlation` is -0.863 : it is not a subtle positional bias but a cut.

The cell’s contribution relative to its siblings is that it shows the truncation is insensitive to where in the tree it happens. The chain reaches the same 37 identifiers by folding right-to-left through seven successive applications, thrashing on the way (its depth-4 and depth-6 merges emit only 300 tokens and 25 items, well under the ceiling, then recover); the flat cell reaches them at the root and then performs four further merges that return the identical blob; the k -ary cell reaches them in two levels. Four fold orders, four depths from 2 to 7, one survivor set — I checked set equality directly and the symmetric difference is empty. Depth cannot cost fidelity when the last node’s budget is the same as every other node’s, because there is no slack for earlier losses to compound into. These four runs are also the sole input to `table_erasure` in `make_tables.py`, which filters to broker-executed incompressible rows, so `tab_erasure` and the macros `\NErasedRanks` (4), `\NTotalRanks` (8) and `\NErasureCorr` (-0.86) are this campaign and nothing else. One correction the paper’s prose needs: it says “five of its eight inputs are missing” where the table says four ranks erased and a fifth reduced to 8%.

Of the flagged findings, the imbalance note and the reattachment note are expected for a binomial tree driven by a shared worker pool, and the model check is the check passing. The genuinely alarming property of this run is one the findings list cannot flag, because nothing went wrong: seven merges, seven successes, one attempt each, zero contract violations, zero hallucinated identifiers, and `n_label_only` is 0 — every identifier that survived carried its correct six-digit serial and four-letter checksum, so `value_retention` and `id_retention` are both 0.3854 exactly. The operator was never wrong and never confabulated. It returned a well-formed consolidated report from which half the population is absent, and no signal anywhere in the protocol distinguishes that from a complete one.

8 Threats to this reading

The executor is real: `executors` records `broker` $\times$ 8 and `worker` $\times$ 18, so this measures model behaviour and not a caricature, unlike the `fid-synth` and `fid-inc-smoke` cells, whose `function` executor implements a drop model by construction. The reading that most needs guarding is therefore the byte-identity of results across the four cells. This run’s final blob is the same object as the `flat` cell’s last five merges and the `kary` cell’s level-0 and level-1 results, and the runs shared one persistent pool of Cursor subagents. I ran the determinism test the contract requires. Two prompt digests are shared across cells — `1586acb2d322` between this run’s `merge:d1` and the chain’s, and `baffd868ca96` between this run’s `merge:d1` and the flat cell’s — and in both cases the results differ (294 against 292 tokens, 288 against 289). Identical input does not give identical output here, so determinism does not explain the identities. But neither does memory need to: the surviving items are verbatim copies of high-entropy input strings emitted in rank order, so once the truncation point is fixed at 37 there is essentially one string that can be returned. Convergence is the parsimonious reading and I cannot exclude pool memory, but here it would have nothing left to explain.

Second, the 11.81-tokens-per-item constant is measured with `tokenizer_exact: false`, so the estimator is the repository’s own approximate counter — the same counter the contract check uses, which is what makes the arithmetic self-consistent even if it is not a real tokeniser’s. A different tokeniser would move the cut by an item or two and would not move the conclusion. The \$0.0500 is likewise modelled from \$3/Mtok in and \$15/Mtok out, not billed.

Third, this is one trial. The latency comparison against the siblings is safe in direction because it is mechanical — three applications on the critical path here against seven for chain and flat and two for k -ary — and this run’s 80.27s against `kary`’s 51.77s is close to the 3:2 ratio the structure predicts. The comparison against the chain’s 634.43s is not safe, because the chain’s per-call generation times

average 89s against this run’s 21.8s for equivalent prompts; that is a pool-side difference, not an algorithmic one, and I would not report $7.9\times$ as a property of the collective.

Finally, two accounting caveats. `fact_retention` scores an identifier as retained wherever it appears in the result text, and the merge prompt’s own instruction line reads “such as [F-3-2]” — an identifier inside the benchmark’s own space at $p \geq 4$ with ≥ 3 items per rank. In the surrogate campaigns that line is copied into every accumulator and inflates the score; here I checked, and rank 3’s survivor is F-3-0 rather than F-3-2, so this run is clean. It remains a latent scoring hazard rather than an error in this measurement. And every `msg.recv` records `admitted: false`, because the collective’s internal receives pass `admit=False`, so context occupancy is 0% and this run says nothing at all about admission pressure despite being the configuration where a root’s context should be under most strain.